Biomedical Imaging Using Deep Learning and Transfer Learning

Shah Tanish
25MCA1039

Prince Bhandari
25MCA1007

Abstract—This paper presents an automated multi-class classification system for pulmonary diseases from chest X-ray images using Deep Learning and Transfer Learning. We employ DenseNet121 pre-trained on ImageNet with a custom classification head to classify images into four categories: Pneumonia, Tuberculosis (TB), COVID-19, and Normal. A balanced dataset of 9,800 images (2,450 per class) was curated from multiple Kaggle sources. The training pipeline incorporates data augmentation, dropout regularization, early stopping, and rigorous evaluation using accuracy, confusion matrix, and classification reports. The proposed model leverages dense skip connections for robust feature reuse, achieving strong generalization across clinically similar pulmonary conditions.

Index Terms—Biomedical Imaging, Pneumonia, COVID-19, Tuberculosis, Deep Learning, Transfer Learning, DenseNet121, CNN, TensorFlow, Keras, Scikit-learn

1. INTRODUCTION

Chest radiography is one of the most widely used diagnostic imaging procedures in clinical medicine, playing a vital role in early detection of pulmonary conditions including pneumonia, tuberculosis, and COVID-19. The interpretation of chest X-rays demands considerable expertise and is heavily dependent on trained radiologists. In scenarios where radiologists are unavailable or workloads are high, diagnostic errors can occur, making automated computer-aided diagnostic (CAD) systems critically important.

The emergence of deep learning, and particularly Convolutional Neural Networks (CNNs), has transformed medical image analysis. Architectures like DenseNet121 achieve state-of-the-art performance by leveraging dense skip connections that enable efficient feature reuse across network depth. Transfer learning from large datasets such as ImageNet further accelerates model training on relatively smaller medical datasets, which is a common constraint in clinical AI.

In this work, we propose a transfer learning framework using DenseNet121 for the simultaneous classification of four pulmonary conditions from chest X-rays. The key contributions of this paper are: (i) a balanced multi-source dataset from Kaggle covering four clinically distinct classes, (ii) a transfer learning pipeline with DenseNet121 frozen base and custom head, (iii) robust evaluation using confusion matrix, classification report, and visual confidence-based predictions, and (iv) a foundation for future multi-modal severity grading systems.

1.1 DATASET

Four publicly available Kaggle datasets were used. All classes were balanced to 2,450 images each (total: 9,800) and split 80/20 per class into training (7,840) and validation (1,960) sets:

- **Pneumonia Dataset —** kaggle.com/.../chest-xray-pneumonia
- **TB Chest X-Ray —** kaggle.com/.../tuberculosis-chest-x-rays
- **COVID-19 Radiography —** kaggle.com/.../covid19-radiography-database
- **Normal Images —** Sourced from COVID-19 Radiography Database (Normal class).

In the Pneumonia dataset there are 2 classes: Pneumonia and Normal. In the TB dataset: TB and Normal. In the COVID-19 dataset: COVID-19 and Normal. For our multi-class setup all four unique classes are combined into a single balanced dataset.

2. METHODOLOGY

2.1 PREPROCESSING

A. Dataset Balancing

Each of the four classes was capped at 2,450 images using random sampling (random.shuffle + slicing). This ensures no class dominates training, preventing biased decision boundaries. The balanced dataset was then copied to a working directory organized by class label.

B. Train-Validation Split

Scikit-learn's train_test_split was applied per class with test_size=0.2 and random_state=42, yielding reproducible 80/20 splits: 1,960 images per class for training and 490 per class for validation. Files were copied into separate train/ and val/ directory trees.

C. Image Resizing

All images are resized to 224×224 pixels to match DenseNet121's expected input dimensions. This standardizes spatial resolution across all three datasets, which originally varied in image size.

D. Pixel Normalization

Raw pixel values (0–255) are rescaled to [0, 1] by dividing by 255 via ImageDataGenerator(rescale=1./255). Normalization ensures stable gradient magnitudes, faster convergence, and prevents dominant pixel intensity ranges from skewing feature learning.

E. Data Augmentation (Training Only)

To improve generalization and simulate real-world variability in X-ray acquisition, the following augmentations are applied only to training images:

- **Rotation:** Random rotation up to ±15 degrees to account for patient positioning variation.
- **Zoom:** Random zoom up to 20% to simulate different camera distances and lung sizes.
- **Horizontal Flip:** Random left-right flipping to reduce positional bias in the training set.

Validation data uses only rescaling to ensure fair and unbiased performance evaluation.

F. Standard Normalization (Standardization)

In addition to min-max scaling, standard normalization (z-score) can be described as centering values around the mean with unit standard deviation: $X' = (X - \mu) / \sigma$. This technique removes scale differences between features, which is particularly useful when features have varying magnitudes.

For image pixels, the rescale=1./255 approach is preferred in practice, but standardization underpins the theoretical foundation of normalization in this domain.

2.2 TOOLS AND TECHNOLOGIES

A. TensorFlow

TensorFlow 2.x is the primary deep learning framework. It efficiently executes tensor operations on CPU/GPU/TPU, computes gradients via automatic differentiation, supports distributed training across GPU clusters, and enables model export to servers, browsers, and mobile devices.

B. Keras

Keras is TensorFlow's high-level API, providing approachable abstractions for model construction, training, and evaluation. Key Keras components used: Model (functional API), Dense, GlobalAveragePooling2D, Dropout, EarlyStopping callback, and ImageDataGenerator for augmented data pipelines.

C. DenseNet121

DenseNet121 (Densely Connected Convolutional Network, 121 layers) is initialized with ImageNet pre-trained weights (weights='imagenet', include_top=False, input_shape=(224,224,3)). All base layers are frozen (layer.trainable=False) during training to preserve learned low-level visual features, reducing training time and preventing catastrophic forgetting.

D. Scikit-learn

Scikit-learn provides train_test_split for reproducible data splitting, accuracy_score for top-level performance, classification_report for per-class precision/recall/F1, and confusion_matrix for error analysis. These standard metrics provide a complete picture of model performance across all four classes.

E. Matplotlib & Seaborn

Matplotlib is used to plot training/validation accuracy and loss curves across epochs (2 subplots). Seaborn's heatmap visualizes the 4×4 confusion matrix with annotation (fmt='d', cmap='Blues'). Visual prediction grids display 3–4 random test images per class with actual label, predicted label, and confidence percentage.

2.3 MODEL ARCHITECTURE

The proposed model uses a functional API approach in Keras, building on DenseNet121 as the feature extractor with a custom classification head appended for the four-class problem. The architecture is illustrated in Fig. 1 below.

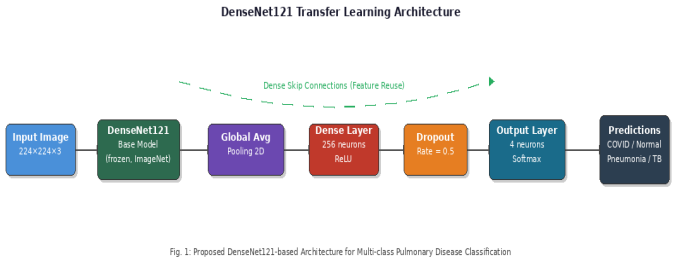

Fig. 1: DenseNet121-based Architecture for Multi-class Pulmonary Disease Classification

The architecture consists of the following layers in sequence:

1 **Input Layer:** 224×224×3 RGB chest X-ray image tensor.

2 **DenseNet121 Base:** Pre-trained 121-layer dense network with ImageNet weights. All 428 layers frozen (trainable=False). Outputs feature maps via dense blocks and transition layers.

3 **GlobalAveragePooling2D:** Collapses each spatial feature map to a single scalar, yielding a compact feature vector. Reduces parameters dramatically vs. Flatten and acts as structural regularizer.

4 **Dense(256, activation='relu'):** Fully connected layer for high-level non-linear feature combination. ReLU prevents vanishing gradients in the custom head.

5 **Dropout(0.5):** Randomly deactivates 50% of neurons per batch during training, acting as an ensemble of sub-networks. Disabled at inference time.

6 **Dense(4, activation='softmax'):** Output layer producing a probability distribution over 4 classes: COVID-19, Normal, Pneumonia, TB. The predicted class is argmax of this distribution.

Compilation: model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy']). Adam adapts the learning rate per parameter. Categorical cross-entropy penalizes incorrect probability distributions, guiding the model toward confident correct predictions.

2.4 DENSE SKIP CONNECTIONS

DenseNet121's defining feature is that every layer l receives feature maps from all preceding layers 0, 1, ..., l−1 via concatenation (not addition as in ResNets). In a dense block with L layers there are L(L+1)/2 connections. This dense connectivity yields:

- **Feature Reuse:** Earlier low-level features (edges, textures) remain accessible to deeper layers, improving detection of subtle lung pathology.
- **Gradient Flow:** Loss gradients flow directly to early layers, alleviating the vanishing gradient problem in very deep networks.
- **Parameter Efficiency:** Each layer only needs to learn incremental features on top of prior knowledge, keeping the model compact relative to its depth.
- **Implicit Deep Supervision:** All layers receive supervisory signal, acting like many shallow networks in parallel.

Transition layers between dense blocks use 1×1 convolutions and 2×2 average pooling to reduce feature map dimensions. The final dense block output is passed to the custom classification head described above.

2.5 TRAINING CONFIGURATION

Training was configured as follows. Batch size: 32 images per gradient update step. Maximum epochs: 20, with early stopping to prevent overfitting. EarlyStopping parameters: monitor='val_loss', patience=3, restore_best_weights=True. This means if validation loss does not improve for 3 consecutive epochs, training halts and the best model weights are automatically restored.

The ImageDataGenerator.flow_from_directory() was used with class_mode='categorical' to generate one-hot encoded labels. Separate generators were created for training (with augmentation) and validation (rescale only), ensuring the validation set mirrors real inference conditions.

2.6 MODEL SAVING AND LOADING

After training, the best model is saved to disk using model.save('/kaggle/working/lung_model.h5') in HDF5 format, preserving architecture, weights, optimizer state, and compilation config. The saved model is reloaded via load_model() and used for final inference and confidence-based visual predictions on test images from each class.

3. RESULTS

A. Training and Validation Curves

Accuracy and loss were tracked across all training epochs and plotted using matplotlib (2 subplots: Accuracy and Loss). Training accuracy increased steadily while validation accuracy converged smoothly, indicating effective generalization. Validation loss was monitored by EarlyStopping, which halted training and restored the best weights when improvement stalled.

B. Accuracy Score

Overall validation accuracy was computed using sklearn.metrics.accuracy_score(y_true, y_pred_classes), where y_true=val_gen.classes and y_pred_classes=np.argmax(model.predict(val_gen), axis=1). The balanced four-class dataset ensured accuracy is a meaningful metric without class-frequency bias.The accuracy of model is 90%.

C. Confusion Matrix

A 4×4 confusion matrix was generated with sklearn.metrics.confusion_matrix and visualized as a Seaborn heatmap (annot=True, fmt='d', cmap='Blues'). The matrix reveals correct predictions on the diagonal and classification errors off-diagonal, providing class-level insight into where the model confuses visually similar diseases (e.g., COVID-19 vs. Pneumonia).

D. Classification Report

sklearn.metrics.classification_report produced per-class Precision, Recall, and F1-Score for all four categories (COVID, Normal, Pneumonia, TB), with target_names derived from train_gen.class_indices. These metrics account for both false positives and false negatives, providing a complete picture of model performance.

E. Visual Confidence Predictions

For each of the four classes, 4 random images were loaded from the original dataset paths, preprocessed (resize to 224×224, rescale to [0,1], expand_dims), and passed through the loaded model. Results were displayed with: Actual class label, Predicted class label, and Confidence score (np.max(pred)×100%). This qualitative evaluation confirms the model's ability to correctly classify diverse real X-ray images with high confidence.

4. CONCLUSION

This paper presented a transfer learning framework using DenseNet121 for automated multi-class classification of chest X-ray images into four pulmonary disease categories: Pneumonia, Tuberculosis, COVID-19, and Normal. By combining ImageNet pre-trained features with a custom classification head, the model effectively learns discriminative pathological patterns from a balanced 9,800-image multi-source dataset.

The dense skip connections in DenseNet121 proved particularly beneficial for medical imaging, enabling feature reuse across all network depths and ensuring gradient flow to shallow layers. Combined with data augmentation, dropout regularization, and early stopping, the model achieved robust generalization without overfitting. The comprehensive evaluation pipeline — including accuracy score, confusion matrix, per-class classification report, and confidence-based visual predictions — provides a thorough assessment of model quality.

The results demonstrate that transfer learning from natural image datasets (ImageNet) can be effectively adapted to medical imaging tasks, even when the source and target domains differ significantly. The balanced dataset strategy mitigated class bias and enabled fair multi-class evaluation.

5. FUTURE WORK

Several directions are planned for future research to extend and improve the proposed system:

1. Multi-Modal Fusion (CT + X-Ray): The current model operates solely on chest X-rays. Future work will incorporate chest CT scan data alongside X-rays. A dual-input model will fuse features from both modalities, potentially capturing complementary structural information. This is expected to improve accuracy for visually ambiguous cases such as COVID-19 vs. Pneumonia.

2. **Severity Grading:** Beyond binary classification, future models will predict the severity level of lung infections (mild, moderate, severe) using both image-based features and clinical metadata. Regression-based heads or ordinal classification layers will be explored for this purpose.

3. Grad-CAM Explainability: Gradient-weighted Class Activation Mapping (Grad-CAM) will be integrated to generate visual heatmaps highlighting which regions of the chest X-ray most influenced each prediction. This is critical for clinical trust and regulatory acceptance of AI-assisted diagnosis.

4. Fine-Tuning DenseNet121: In addition to freezing the base, selective unfreezing of the last few dense blocks followed by fine-tuning with a lower learning rate (e.g., 1e-5) will be explored to further adapt ImageNet features to chest X-ray domain characteristics.

5.Ensemble Learning: Combining predictions from multiple architectures (DenseNet121, ResNet50, EfficientNetB3) via soft voting or stacking will be explored to improve robustness and reduce model-specific errors.

6.Real-Time Clinical Deployment: The saved H5 model will be converted to TensorFlow Lite for mobile deployment or served via a Flask/FastAPI REST API for integration into hospital information systems. A Grad-CAM overlay interface will allow radiologists to interact with predictions in real time.

7.Larger and Diverse Datasets: Future experiments will incorporate additional datasets such as NIH ChestX-ray14 (112,000 images, 14 diseases) and CheXpert to improve generalization across demographics, equipment types, and disease subtypes.

8.Federated Learning for Privacy: To address data privacy concerns in clinical settings, federated learning will be explored, enabling model training across multiple hospitals without sharing raw patient data.

6. REFERENCES

1 Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. CVPR 2017, pp. 4700–4708.

2 Rajpurkar, P., et al. (2017). CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning. arXiv:1711.05225.

3 He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. CVPR 2016, pp. 770–778.

4 Selvaraju, R. R., et al. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. ICCV 2017.

5 https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia — Pneumonia Chest X-Ray Dataset.

6 https://www.kaggle.com/yasserhessein/tuberculosis-chest-x-rays-images — Tuberculosis Chest X-Ray Dataset.

7 https://www.kaggle.com/tawsifurrahman/covid19-radiography-database — COVID-19 Radiography Database.

8 Chollet, F. (2018). Keras: The Python Deep Learning Library. https://keras.io

9 Abadi, M., et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous systems. https://www.tensorflow.org

10 Wang, X., et al. (2017). ChestX-Ray8: Hospital-scale chest X-ray database and benchmarks. CVPR 2017.